Crash-Proof Applications

TOM WHEELER, Temporal Technologies, USA

For decades, ACID transactions have protected *data* from the effects of a crash, but developers have lacked an abstraction to protect their *running applications*. When the process hosting an execution terminates, the values assigned to variables disappear, along with any record of which steps have completed and which have not. This may cause the application to behave incorrectly or repeat completed work upon restart.

Architectural shifts during the past twenty years, including the rise of cloud computing and microservices, have made this problem more urgent by transforming business applications into distributed systems. Simply put, more machines mean more opportunities for things to go wrong. Earlier approaches to fault tolerance established the right ideas, but most remain out of reach because they require specialized hardware, unfamiliar languages, or programming models that are a poor fit for business applications. Durable Execution is practical where many earlier approaches were not: it can be implemented in nearly any general-purpose programming language and run on commodity hardware.

Durable Execution allows an application to survive process termination, system restarts, and machine failures without losing state or unnecessarily duplicating completed work. It suits a wide range of use cases, including order management, payment processing, travel bookings, financial compliance workflows, account provisioning, data pipelines, cloud infrastructure deployment, and agentic AI. Organizations are increasingly adopting it for business-critical production systems, with many reporting dramatic improvements in application reliability and developer productivity. Through illustrative examples, this article explains Durable Execution, describes the history replay technique commonly used to provide it, examines its implications for application design and performance, and summarizes reports from the field.

CCS Concepts: • **Software and its engineering** → **Software fault tolerance**; *Checkpoint / restart*; *Distributed systems organizing principles*; • **Computer systems organization** → *Reliability*.

Additional Key Words and Phrases: durable execution, fault tolerance, workflow systems, history replay, distributed systems

1 The Perils of Unexpected Termination

To understand Durable Execution, first examine the inherent volatility of ordinary execution.

Execution occurs within a process, created by an operating system, and ultimately runs on a physical machine. Problems at any level can result in a crash. An unhandled exception in application code is one example. Crashes may also stem from problems outside the developer's control, such as software defects in a library dependency, language runtime, or operating system kernel. Administrator mistakes, such as misconfigurations or unplanned reboots, can also terminate executions, as can hardware failures and power outages [16].

ACID transactions make an application's data durable, but do nothing to protect the application's internal state. Consider the following naive Python function, which uses two API calls to move $100 between accounts at different banks.

NOTE: Complete code for all examples in this article can be found at https://github.com/tomwheeler/durable-execution-cacm-code

Author's Contact Information: Tom Wheeler, tom@temporal.io, Temporal Technologies, Bellevue, Washington, USA.

2026. Manuscript submitted to ACM

```
def transfer_money():
    bank_one = BankingService("api.miami-bank.example.com")
    confirmation1 = bank_one.withdraw("A123", 100)

    bank_two = BankingService("api.seattle-bank.example.com")
    confirmation2 = bank_two.deposit("B789", 100)

    return generate_success_message(confirmation1, confirmation2)
```

Even if both banks' services are operational, correctness depends on completion of both steps. If execution terminates between the calls to `withdraw` and `deposit`, $100 will be lost, having been debited from the first account yet never credited to the second. This behavior is clearly unacceptable.

The underlying problem is that execution is tied to a process. The values currently assigned to variables and an awareness of which steps have already completed are transient. When that process exits, they disappear. Unless the developer has taken steps to handle this situation, restarting the execution will return to the beginning, repeating work that was previously completed, instead of resuming from the point where the crash occurred. This duplication is inefficient, but more importantly, may lead to incorrect behavior. In this example, restarting the application would repeat the withdrawal, so even if the deposit did succeed during this second attempt, the first account would still be missing $100.

Many consumers have experienced the effects of this problem firsthand, whether charged multiple times for a single purchase or sent duplicate confirmations after placing an online order.

Why not solve this with ACID transactions across the board? The ubiquity of cloud computing and a widespread reliance on third-party APIs make this untenable. A generation of application developers has become distributed systems developers without many realizing it. Their mandate is to solve business problems, often by composing applications that include components they do not control. They need reliability, but as Pat Helland points out [20], "Most developers simply don't have access to robust systems offering scalable distributed transactions." Durable Execution provides an alternative that is quickly gaining industry adoption because it addresses these developers' needs and is practical in the environments where their applications run.

2 Existing Approaches and Their Limits

Crashes are inevitable. Broadly, there are two approaches for mitigating them.

The first aims to prevent process termination through hardware-level fault tolerance. RAID, redundant power supplies, and rackmount uninterruptible power supplies are typical for commodity servers in a modern data center. High-end systems, such as mainframes and enterprise-class UNIX servers, often go further, offering hot-swappable processors and memory modules.

The second seeks fault tolerance through software, typically via a rollback-recovery technique [12] such as application checkpointing.

Hardware-based approaches may *reduce* terminations caused by hardware failures or power outages, but can never completely *eliminate* them. Furthermore, they cannot protect against crashes caused by software defects.

Software-based approaches provide broader protection, since they can guard against crashes from both hardware failure and software defects. This protection also has a cost, since checkpointing consumes time and resources.

Neither suffices on its own. In practice, systems often combine the two: redundant hardware to make crashes rare, and checkpointing to recover from the ones that still occur.

Checkpointing frameworks, such as BLCR [17] and VeloC [28], have existed for many years, yet have seen little adoption outside high-performance computing and batch environments. One cited reason is that they are typically system-level tools [3]; another explanation is that they simply weren't designed with business application developers in mind. They are often tightly coupled to parallel computing libraries (such as MPI) and may mandate a certain language (such as C or FORTRAN), so they don't necessarily reflect the infrastructure and skills of those teams.

Consequently, business application developers routinely implement their own homegrown approaches to checkpointing. This typically involves using a transactional database, copying values from within their running applications and storing them in the database for safekeeping. If a crash does occur, they attempt to recover by loading those values from the database and updating variables within their applications. This can be a significant amount of work. Depending on the programming language and database being used, it is often made worse by the object-relational impedance mismatch, a problem Ted Neward famously called the "Vietnam of Computer Science" [27]. One study found that this mismatch can increase the time it takes to write such code by one-third or more [40].

More than forty years ago, Anita Borg and her colleagues at Auragen argued that none of this should be necessary [6]. They held that fault tolerance should be transparent—provided by a platform rather than the application—with automatic detection and recovery. The decades since have advanced toward that vision. A system called Ken gave distributed applications transparent rollback-recovery, letting them be written as though tolerated failures could not occur [39]. Orleans's virtual actors demonstrated that fault tolerance could be built into the programming model rather than repeatedly implemented in application code [5]. MillWheel delivered exactly-once processing for streaming data at Google scale, handling retries and deduplication otherwise left to developers [2]. In each case, the burden shifted from the application to the platform.

3 What Is Durable Execution?

Although crashes are inevitable, Durable Execution enables an application to overcome them with no loss of state and without repeating steps in a way that would change its outcome.

This is made possible by an abstraction that insulates an application from crashes, allowing execution to continue despite them. A Durable Execution platform is the system that provides this abstraction. A variety of Durable Execution platforms are available today, both commercial and open source. These include Temporal [21], Cadence [8], Restate [31], Inngest [22], DBOS [10], Hatchet [18], and Trigger.dev [35].

Most of these platforms refer to the abstraction as a *workflow*. Please set aside any preconceived notions about this term. Unlike many systems, where workflows are defined through diagrams or configuration files, these platforms follow a "workflows-as-code" approach. In this context, a workflow is simply a function or method that has the ability to survive a crash, with no loss of state. This means that values assigned to variables in this function are automatically persisted, and if a crash occurs, those values will be the same following recovery as they were before the crash. As a result, an application built on Durable Execution does not need to save and restore that state itself.

3.1 Durable Execution Can Span Multiple Processes over Time

Normally, execution is bound to a single process on a single machine. When that process terminates, the execution terminates along with it. Durable Execution breaks this one-to-one mapping between the execution and the process. It

virtualizes execution, giving it a logical form independent of any particular process. It can span a *series* of processes over time, potentially across multiple machines, seamlessly from the developer's point of view.

An example will help to explain this concept. Imagine a business operation, such as processing an order for electronic books, which involves five steps:

(1) Calculate the price of all books in the order
(2) Validate the coupon code, if available
(3) Calculate the discounted total, if the coupon is valid
(4) Charge the customer's payment card
(5) Send an email confirmation with a download link

Now imagine the first two steps complete successfully, but afterward the application immediately crashes. Execution continues in a new process. Recovery fully reconstructs the previous state, including all variable values and all previously created threads. Once this is complete, execution resumes with the third step. Say this step completes successfully, but execution crashes again, this time due to a hardware failure. In this case, progress continues in a new process on a different machine, which then completes successfully.

Figure 1 illustrates two views of this execution. The physical view, at the top, shows execution spanning three processes on two machines, crashing twice before completion. These details are transparent to the developer, who experiences the logical view at the bottom. From their perspective, each step completed successfully and the application achieved its goal without issue.

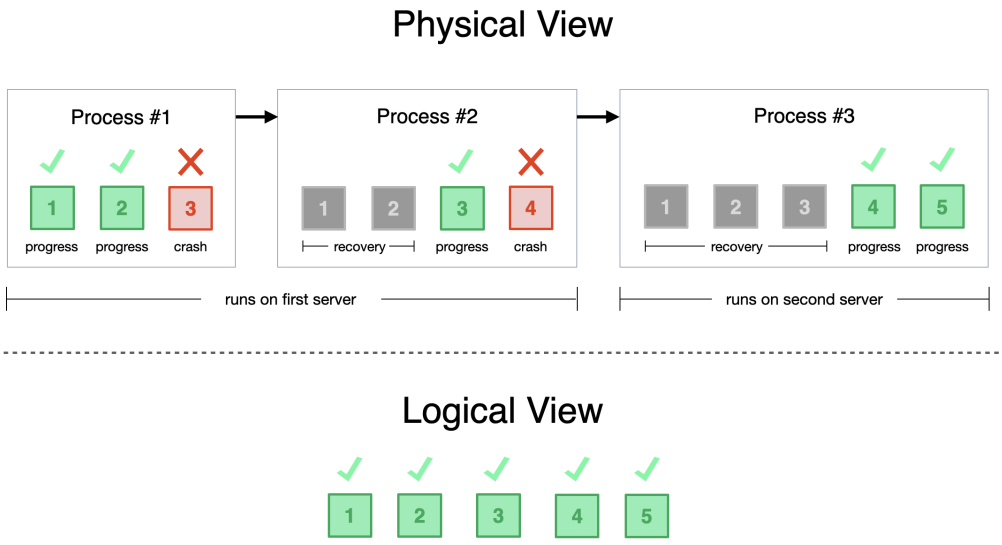

Fig. 1. Physical vs. Logical View

This protects an application that's moving forward from a crash, but also protects one that's moving backward. Consider the order processing workflow described above. If a crash occurs just before the customer's card is charged, the application recovers and resumes, then charges the customer. What if the book they purchased is no longer available? Since it cannot deliver what the customer paid for, it must refund the purchase price. Compensating for work that

cannot be completed is the basis of the saga pattern [15], and Durable Execution ensures those compensating steps survive a crash just like the ones they reverse.

4 How Durable Execution Works

Durable Execution could conceivably be implemented in several ways. The first that comes to mind for most developers involves snapshotting an application's execution state, including its memory space, CPU registers, and open file descriptors. However, this approach is not widely used among Durable Execution platforms. Trigger.dev, which uses the CRIU [34] framework, is the only listed platform that works this way.

A more common approach, used by Temporal, Cadence, Restate, and others, is history replay, known in the literature as deterministic record-and-replay [30]. Because it depends only on recording and replaying an execution's steps, it is platform-independent, language-agnostic, and does not require specialized hardware. It is supported by a formal proof [7] and used at massive scale in production. As of May 2026, Temporal's cloud service manages nearly thirty-five billion workflow executions per day [1]. This does not include workflow executions managed by the more than five hundred thousand self-hosted deployments in active use during a typical week [14].

Terminology and implementation details vary across platforms, and covering all of them is impractical. Except as noted, the explanations and examples that follow are based on Temporal, because it is widely used, open source (MIT license), well documented, and supports many programming languages.

4.1 History Replay

History replay is based on the idea that it's unnecessary to checkpoint the complete state of the application; that's often too much data. One merely needs to store the data required to *reconstruct* that state on demand, which is generally far smaller. This approach requires workflow code to be deterministic. Non-deterministic operations and externally visible side effects must be isolated in separate functions, which Temporal calls *activities*. Note that while the internal state of the workflow function is durable, the same is not true for activity functions. The workflow function's typical role is to orchestrate an overall goal, such as processing an order, while an activity function is used to implement a specific step within that sequence, such as charging the customer or sending an email notification. If the activity function crashes or fails, it is simply retried.

Recall the e-book order processing described earlier, which involved five steps:

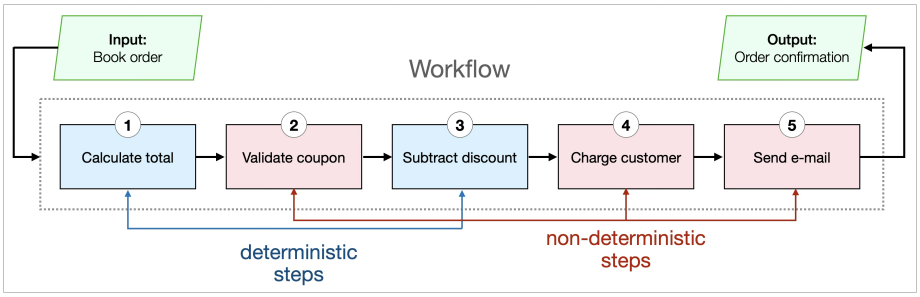

Fig. 2. Workflow steps for the e-book order process

Steps 1 and 3 are deterministic, as they involve only basic arithmetic and do not interact with any external system. Given the same input, every invocation always produces the same result. Code for these steps may remain within the workflow function.

Steps 2, 4, and 5 are non-deterministic, since they could behave differently from one invocation to the next. For example, step 2 might behave differently because the list of valid coupon codes changes as new offers appear and older ones expire. Step 4 depends on the payment service, which may fail to respond if it becomes overloaded. Similarly, step 5 may vary if the email server encounters intermittent connection failures from faulty network infrastructure or delays in DNS propagation.

Code for steps 2, 4, and 5 must be moved into activity functions. These are not invoked directly by the workflow code, but are referenced by it and invoked during its execution, as shown in Figure 3. This indirection allows the platform's language-specific application support library (known as an SDK) to intercept calls, schedule execution of those functions, and monitor their lifecycle.

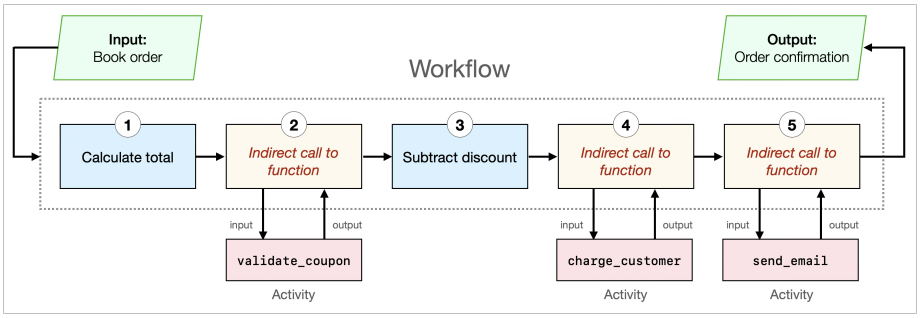

Fig. 3. Indirection between a workflow and its activities

The following workflow, written in Python, implements the e-book order process described earlier. It includes the deterministic code for calculating the pre- and post-discount prices, as well as code for orchestrating activities that carry out the non-deterministic operations. Each of those activities is invoked indirectly via `workflow.execute_activity`, which is provided by the Temporal SDK. Each call to `workflow.execute_activity` here includes three arguments: the activity type (function name), data passed as input to the activity, and a timeout that limits how long the activity is allowed to run. The `try/except` block near the end demonstrates the saga pattern by compensating the customer with a refund if their book order was charged but not fulfilled.

```python
@workflow.defn
class BookOrderWorkflow:
    @workflow.run
    async def process_order(self, order: BookOrder) -> Receipt:
        total_price = sum(item.price for item in order.items)

        if order.coupon_code:
            discount_percent = await workflow.execute_activity(
                validate_coupon, order.coupon_code,
                start_to_close_timeout=timedelta(seconds=10),
            )
            total_price -= round(total_price * discount_percent / 100)

        confirmation = await workflow.execute_activity(
            charge_customer, PaymentInput(order.card_number, total_price),
            start_to_close_timeout=timedelta(seconds=30),
        )

        try:
            await workflow.execute_activity(
                send_email, order.email_addr,
                start_to_close_timeout=timedelta(seconds=15),
            )
        except ActivityError:  # saga pattern: compensate upon failure
            confirmation = await workflow.execute_activity(
                refund_customer, PaymentInput(order.card_number, total_price),
                start_to_close_timeout=timedelta(seconds=30),
            )

        return Receipt(order.order_number, total_price, confirmation)
```

4.1.1 Event History. The platform tracks the execution of both the workflow function and the activity functions it references. As the workflow execution progresses, the platform updates an append-only log with details of the invocation, execution status, and result (or error message) returned by those functions. That log is known as the *event history*, and it contains all of the data needed for recovery. The following table illustrates some of the data that would be stored in the event history for the e-book order workflow.

Table 1. Selected events in the e-book order workflow history

Event name	Associated function	Value stored in event
`WorkflowExecutionStarted`	`BookOrderWorkflow.run`	Input to the workflow
`ActivityTaskScheduled`	`validate_coupon`	Input to the activity
`ActivityTaskCompleted`	`validate_coupon`	Output of the activity
`ActivityTaskScheduled`	`charge_customer`	Input to the activity
`ActivityTaskCompleted`	`charge_customer`	Output of the activity
`ActivityTaskScheduled`	`send_email`	Input to the activity
`ActivityTaskCompleted`	`send_email`	Output of the activity
`WorkflowExecutionCompleted`	`BookOrderWorkflow.run`	Output of the workflow

Following a crash, the application recovers by starting a new process and then uses this event history to guide reconstruction of the pre-crash state. It re-executes the workflow code, which is deterministic, so it follows the same path as it did earlier. When it reaches a call to an activity function, it checks the event history to determine whether the call completed before the crash. If it did, it uses the result stored in the event history instead of executing the activity again.

It continues this process until it reaches an activity whose completion isn't listed in the event history. At that point, recovery is complete: all workflow variables, threads, and coroutines are in the same state as immediately before the crash. Execution then continues, running remaining code as it normally would. Recovery happens automatically and is transparent to developers.

The event history is written to durable storage, typically a transactional database such as PostgreSQL. Durable Execution platforms tend to be pessimistic, so if that storage is unavailable, workflow execution waits instead of risking a lost step. It is therefore important to choose reliable storage and size it appropriately for the load.

Although the primary purpose of the event history is to enable recovery, it also provides an audit trail of what took place. In addition to a tabular view, which includes precise timestamps and details about each event, Temporal's web-based user interface includes a timeline visualization showing the sequence of events. Figure 4 shows an example of this for the e-book order workflow. The three green lines at the bottom each represent an activity, while the long green line above all of them represents the workflow. The input passed to the workflow function and the result it produced are shown above the timeline.

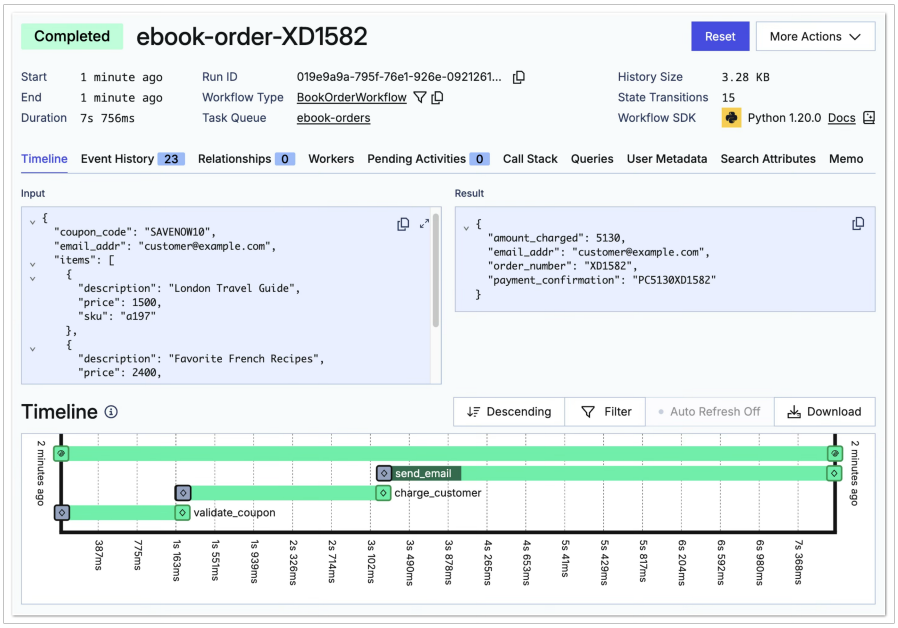

Fig. 4. Timeline view of event history in Temporal's Web UI

The event-sourcing approach used for history creation preserves the inputs, function names, and results of calls that led to the current state. The history is typically retained for many days after execution completes, so it also serves as a valuable debugging aid. In fact, Temporal and some other platforms let developers replay a past execution in a debugger

to quickly reproduce, diagnose, and fix a problem observed in production. Many teams protect against regressions by writing automated tests that replay executions related to bugs they have fixed.

Upon learning about the event history, many developers naturally wonder how to handle sensitive data, such as financial account numbers, medical patient information, or other personally identifiable information (PII). The history replay technique does not rely on the platform being able to interpret the data, so the application can encrypt sensitive data before it is stored in the event history. Details vary among Durable Execution platforms, but in the case of Temporal, this is done using an API for transforming application data before it is transmitted to the platform and reversing those transformations when the data is received [38]. The developer selects the cipher and manages the encryption key. The platform receives only encrypted data and, without access to the key, is unable to decrypt it. Since this API allows arbitrary transformations, organizations with demanding security requirements can instead replace sensitive data with token values. In that case, sensitive data is never stored by the platform, even in encrypted form.

4.1.2 The Deterministic Constraint. History replay requires workflow code to be deterministic. Developers sometimes misinterpret this constraint, concluding that Durable Execution isn't suitable for applications involving LLMs or AI agents, since those are non-deterministic. What they fail to recognize is that the constraint does not limit what the application can do; it merely specifies which type of function should contain the code to do it.

Interactions with external systems are inherently non-deterministic, since their results depend on the availability and behavior of those systems at that moment. It's the same whether the application queries a database, invokes a microservice, or calls a large language model: all of them could behave differently each time. Consequently, calls to external systems must take place within activities.

Once an activity's result is recorded to history, replay does not execute that activity again; it returns the stored value. Non-deterministic operations therefore behave deterministically during recovery, whether they involve databases, payment processors, LLMs, or any other external system.

5 Performance

Durable Execution's benefits come with a performance cost. It takes two forms: recovery time and increased latency. Reducing the overhead is an area of active research [25].

The latency stems from updating the event history at each significant step during execution. The amount varies based on many factors, such as the performance of the storage system and the network used to access it. One study of an AI agent using Temporal measured this at roughly 80 milliseconds per interaction, about 11% of total end-to-end latency with an LLM, which could be reduced by tuning the default configuration [11].

For a simple workflow, recovery can complete in a fraction of a second, but that time grows with the number of events and the data stored in each. Platforms therefore limit both aspects to maintain good performance. Recovery time bears no relationship to the execution's running time, and since completed activities are not repeated, Durable Execution can save significant time compared to a system that starts over after a crash. This is especially beneficial for jobs with long-running steps and strict SLAs, such as nightly ETL jobs or model training.

Durable Execution is not a good fit at present for high-frequency trading and other extremely latency-sensitive applications. It is viable for applications that prioritize fault tolerance, as evidenced by financial firms that have adopted it for use cases ranging from payments [23] to loan origination [32]. For existing applications with homegrown checkpointing and recovery logic, migrating to a Durable Execution platform can even improve performance, since such platforms may include optimizations the application developers overlooked or lacked the resources to implement.

OpenAI exemplifies the scale of Durable Execution in production for a single application. ChatGPT uses Temporal workflows for its image-generation feature [29], and in May 2026 OpenAI's VP of Applied Infrastructure stated that the system generates around one billion images per week [37].

6 Failure in External Systems

Durable Execution refers to the application's ability to survive a crash, but crashes are not the only problem developers face. Applications must often interact with systems that are unavailable, unstable, or unresponsive. To help developers build reliable applications in an unreliable world, most Durable Execution platforms also provide built-in retries and timeouts.

Engineers at Tandem Computers observed [4] that most failures are transient, and this is even more true in the modern era of LLMs, microservices, and cloud computing. When a call fails due to a network glitch, temporary overload, or service outage, repeating it after a short delay is usually enough to resolve it. This requires no knowledge of the application, so the platform handles it at runtime without developer interaction.

Developers can customize retry behavior per activity execution, specifying parameters such as initial delay, backoff coefficient, and non-retryable error types (such as one that a payment processor returns for an expired credit card). Although standalone retry libraries exist for most languages, retries provided by a Durable Execution platform have an important advantage: state associated with retry attempts is not lost if the process crashes.

An unavoidable consequence of retrying is that an activity may execute more than once. This is expected in most cases, since it is retried upon failure or timeout. However, there's also an edge case to consider. If an activity succeeds but its completion isn't recorded in the event history, the platform won't know, and the activity will execute again if retries are allowed.

Idempotence is the recommended defense. Payment providers and many other services are often designed to support it. By including a token known as an idempotency key, a service can detect and ignore duplicate calls, so repeated invocations have no effect. As Pat Helland explained, idempotence is essential to any reliable system involving retries [19]. Making activities idempotent is considered a best practice.

6.1 Recovering from Bugs

An interesting aspect of a Durable Execution platform is that it lets developers fix bugs in a running production system.

Imagine an activity calls a REST API, but its code specifies the wrong URL. The activity fails when it attempts the call, and every retry likewise fails. The developer can fix the bug by correcting the URL in the activity function, deploying the updated code, and restarting the application. The outcome is the same as if the application had crashed: after recovery, the failing activity is reattempted with the updated code and can succeed.

7 Time Is No Longer the Enemy

Durable Execution lets an application run as long as necessary to achieve its goal. An execution can outlive the process and machine that started it, running for months or years. Because its state survives a crash, a long-running process can keep its data in ordinary variables rather than copying it to a database for safekeeping. With Durable Execution, developers no longer need an application database or the code to interact with one simply to guard against a crash.

A recurring, long-lived process that once required an external scheduler and a queue can be written as an ordinary loop. The payroll workflow below, written in Python, illustrates this. It deposits the employee's earnings every two weeks for as long as they remain employed. Signals allow a caller to change the state of a running workflow; in this

case, one signal ends employment, while another changes the amount the employee is paid. Queries allow a caller to retrieve values from the workflow; in this example, there are two. One query returns the employee's current payroll amount while the other returns the cumulative amount paid during their entire employment period.

```
@workflow.defn
class PayrollWorkflow:
    def __init__(self):
        self.is_employed = True
        self.pay_rate = 0
        self.cumulative_pay = 0

    @workflow.run
    async def run(self, account: str, pay_rate: int) -> None:
        self.pay_rate = pay_rate

        while self.is_employed:
            amount = self.pay_rate
            await workflow.execute_activity(
                deposit, args=[account, amount],
                start_to_close_timeout=timedelta(seconds=30),
            )
            self.cumulative_pay += amount
            await workflow.sleep(timedelta(days=14))

    @workflow.signal
    def end_employment(self):
        self.is_employed = False

    @workflow.signal
    def set_pay_rate(self, pay_rate: int):
        self.pay_rate = pay_rate

    @workflow.query
    def get_pay_rate(self) -> int:
        return self.pay_rate

    @workflow.query
    def get_cumulative_amount(self) -> int:
        return self.cumulative_pay
```

This loop runs as long as the person remains on the payroll, which could be many years. With each payroll cycle, a handful of entries are added to the event history. The platform limits how large the history may grow, which keeps recovery fast and storage bounded. Assuming a two-week payroll cycle and an annual pay increase, this example could run for more than 30 years before the history grows large enough to cause a warning in Temporal and more than 150 years before it is terminated for exceeding the limit. Even so, the platform can continue an execution in a new run with an empty event history, and since these can be chained together, a workflow could effectively run in perpetuity.

8 Why Adoption of Durable Execution Is Growing

The notion of platform-based fault tolerance isn't new. Why is Durable Execution gaining adoption in industry while previous approaches struggled? Earlier approaches asked too much of developers.

Fault tolerance at Tandem depended on specialized, redundant hardware [4]. Similarly, JUSTDO logging enables crash-interrupted code to resume, but requires a system with persistent memory and processor caches [24]. Many checkpointing frameworks mandated an unfamiliar language or difficult programming model. For teams building ordinary business software, these requirements can be barriers.

Conversely, applications built on a Durable Execution platform can run on commodity hardware and be written in a mainstream language the developers already use, such as Python, TypeScript, Go, Java, C#, Ruby, or Rust. Durable Execution meets developers where they are and supports the types of applications they build.

9 Reports from the Field

The gains are not hypothetical. Uber's developers, surveyed after adopting Cadence, reported building features with 40% less code, and Cadence now runs more than a thousand services there and handles over twelve billion executions a month [36]. ANZ Bank delivered a loan-origination system eight times faster after adopting Temporal [32], and Maersk cut the time to add a feature from as much as eighty days to as few as five [33]. Forty-five percent of Cash App's developers report saving weeks or months of engineering time [13].

Another striking example comes from Netflix, which began using Temporal in 2021. Engineers working on Spinnaker, the multi-cloud continuous delivery platform behind the vast majority of the company's software deployments, reported that rearchitecting it with Temporal reduced transient deployment failures from 4% to just 0.0001% [26].

10 Conclusion

Platforms elevate developers by hiding reality behind a useful abstraction. For example, the UNIX filesystem enables users to think about storage in terms of files instead of disk sectors. Databases build on this with a higher-level abstraction, enabling developers to think in terms of fields, records, and tables instead of file structures and serialization formats.

Durable Execution platforms elevate developers by providing an abstraction for fault tolerance. Because the platform virtualizes execution, the application's state and progress outlive any single process. The logical execution can span a series of processes and machines over whatever duration is necessary to achieve the goal.

A Durable Execution platform can substantially reduce the application code required to achieve a goal. Developers spend significant effort writing code to detect and handle failures. In fact, computer science professor Flaviu Cristian reported that such code often accounts for more than two-thirds of all code in a production system [9]. By shifting that responsibility from the application to the platform, developers can create reliable applications that are faster to develop and easier to understand, and that require less effort to maintain.

ACID transactions make application data durable, while Durable Execution protects the application's internal state. The platforms that provide it let developers focus on what the application should achieve instead of fixating on the problems that might occur along the way. This fulfills the vision that Anita Borg and her colleagues described more than forty years ago: transparent fault tolerance provided by a platform instead of being reinvented inside each application.

References

[1] Samar Abbas. 2026. Replay 2026 Opening Keynote with Samar Abbas. YouTube video, Replay Conference 2026. Retrieved June 17, 2026 from https://www.youtube.com/watch?v=BxEB7Y2U9oU.

[2] Tyler Akidau, Alex Balikov, Kaya Bekiroğlu, Slava Chernyak, Josh Haberman, Reuven Lax, Sam McVeety, Daniel Mills, Paul Nordstrom, and Sam Whittle. 2013. MillWheel: Fault-Tolerant Stream Processing at Internet Scale. *Proceedings of the VLDB Endowment* 6, 11 (2013), 1033–1044.

[3] Jason Ansel, Kapil Arya, and Gene Cooperman. 2009. DMTCP: Transparent checkpointing for cluster computations and the desktop. In *Proceedings of the 2009 IEEE International Symposium on Parallel & Distributed Processing (IPDPS '09)*. IEEE Computer Society, Washington, DC, USA, 1–12. doi:10.1109/IPDPS.2009.5161063

[4] Joel F. Bartlett, Jim Gray, and Bob Horst. 1986. *Fault Tolerance in Tandem Computer Systems.* Technical Report 86.2. Tandem Computers, Cupertino, CA.

[5] Philip A. Bernstein, Sergey Bykov, Alan Geller, Gabriel Kliot, and Jorgen Thelin. 2014. *Orleans: Distributed Virtual Actors for Programmability and Scalability.* Technical Report MSR-TR-2014-41. Microsoft Research.

[6] Anita Borg, Jim Baumbach, and Sam Glazer. 1983. A message system supporting fault tolerance. *SIGOPS Oper. Syst. Rev.* 17, 5 (Oct. 1983), 90–99. doi:10.1145/800217.806617

[7] S. Burckhardt, C. Gillum, D. Justo, K. Kallas, C. McMahon, and C. S. Meiklejohn. 2021. Durable functions: semantics for stateful serverless. *Proceedings of the ACM on Programming Languages* 5, OOPSLA, Article 133 (Oct. 2021), 27 pages. doi:10.1145/3485510

[8] Cadence. 2026. Cadence. https://cadenceworkflow.io.

[9] F. Cristian. 1995. Exception Handling and Tolerance of Software Faults. In *Software Fault Tolerance*, Michael R. Lyu (Ed.). John Wiley & Sons Ltd., Chapter 4, 81–82.

[10] DBOS. 2026. DBOS. https://dbos.dev.

[11] Stenio de Lima Ferreira. 2026. Durable Execution for AI Agents: A Design Pattern for Fault-Tolerant Agent Loops. Online paper. Retrieved June 6, 2026 from https://agtnyc.com.

[12] E. N. M. Elnozahy, L. Alvisi, Y.-M. Wang, and D. B. Johnson. 2002. A survey of rollback-recovery protocols in message-passing systems. *ACM Comput. Surv.* 34, 3 (Sept. 2002), 375–408. doi:10.1145/568522.568525

[13] Nick Esposito and Riley Pruitt. 2024. Great tech is not enough: building trust to get the most out of Temporal. YouTube video, Replay Conference 2024. Retrieved June 17, 2026 from https://www.youtube.com/watch?v=Eq-j0wPR2Es.

[14] Maxim Fateev, Preeti Somal, and Cornelia Davis. 2026. Replay 2026 Closing Keynote with Maxim Fateev ft. Preeti Somal and Cornelia Davis. YouTube video, Replay Conference 2026. Retrieved June 17, 2026 from https://www.youtube.com/watch?v=Rx0reHWvttY.

[15] Hector Garcia-Molina and Kenneth Salem. 1987. Sagas. In *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data (SIGMOD '87)*. ACM, 249–259. doi:10.1145/38713.38742

[16] Jim Gray. 1986. Why do computers stop and what can be done about it?. In *Proceedings of the 5th Symposium on Reliability in Distributed Software and Database Systems*. 3–12.

[17] Paul Hargrove and Jason Duell. 2006. Berkeley Lab Checkpoint/Restart (BLCR) for Linux clusters. In *Journal of Physics: Conference Series (SciDAC 2006)*, Vol. 46. IOP Publishing Ltd, 067. doi:10.1088/1742-6596/46/1/067

[18] Hatchet. 2026. Hatchet. https://hatchet.run.

[19] Pat Helland. 2012. Idempotence is not a medical condition. *Commun. ACM* 55, 5 (May 2012), 56–65. doi:10.1145/2160718.2160734

[20] Pat Helland. 2016. Life beyond distributed transactions: an apostate's opinion. *ACM Queue* 14, 5 (Oct. 2016), 69–98. doi:10.1145/3012426.3025012

[21] Temporal Technologies Inc. 2026. Temporal. https://temporal.io.

[22] Inngest. 2026. Inngest. https://inngest.com.

[23] Rajesh Iyer. 2024. JPMC: A Payments Modernization Journey using Temporal. YouTube video, Replay 2024. Retrieved June 26, 2026 from https://www.youtube.com/watch?v=kuwo9m-K9Bg.

[24] Joseph Izraelevitz, Terence Kelly, and Aasheesh Kolli. 2016. Failure-atomic persistent memory updates via JUSTDO logging. In *Proceedings of the 21st International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '16)*. ACM, New York, NY, USA, 427–440. doi:10.1145/2872362.2872410

[25] Tianyu Li, Badrish Chandramouli, Philip A. Bernstein, and Samuel Madden. 2024. Distributed Speculative Execution for Resilient Cloud Applications. *arXiv preprint arXiv:2412.13314* (2024).

[26] Netflix. 2025. How Temporal Powers Reliable Cloud Operations at Netflix. Retrieved May 19, 2026 from https://netflixtechblog.com.

[27] Ted Neward. 2006. The Vietnam of Computer Science. Retrieved May 22, 2026 from https://odbms.org.

[28] Bogdan Nicolae, Adam Moody, Elsa Gonsiorowski, Kathryn Mohror, and Franck Cappello. 2019. VeloC: Towards High Performance Adaptive Asynchronous Checkpointing at Large Scale. In *2019 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 911–920. doi:10.1109/IPDPS.2019.00099

[29] Gergely Orosz. 2025. Building, launching, and scaling ChatGPT Images. The Pragmatic Engineer. Retrieved June 19, 2026 from https://newsletter.pragmaticengineer.com/p/chatgpt-images.

[30] Austen Quinn and Peter Alvaro. 2025. Deterministic record-and-replay. *Commun. ACM* 68, 5 (April 2025), 32–34. doi:10.1145/3724381

[31] Restate. 2026. Restate. https://restate.dev.

[32] Temporal Technologies Inc. 2026. How ANZ Bank accelerated home loan origination with Temporal. https://temporal.io/resources/case-studies/anz-story.
[33] Temporal Technologies Inc. 2026. How Maersk Built a “Time Machine” for the Logistics Industry with Temporal. https://temporal.io/resources/case-studies/maersk.
[34] A. Tošić. 2024. Run-time Application Migration using Checkpoint/Restore In Userspace. *Journal of Web Engineering* 23, 5 (Aug. 2024), 735–748.
[35] Trigger.dev. 2026. Trigger.dev. https://trigger.dev.
[36] Uber. 2023. Announcing Cadence 1.0: The Powerful Workflow Platform Built for Scale and Reliability. Retrieved November 30, 2025 from https://uber.com.
[37] Venkat Venkataramani. 2026. OpenAI @ Replay 2026: Temporal, OpenAI, and the Future of Agentic AI with Venkat Venkataramani. YouTube video, Replay 2026. Retrieved June 19, 2026 from https://www.youtube.com/watch?v=65CCmktUA8I.
[38] Tom Wheeler and Joshua Smith. 2025. How to Protect Sensitive Data in a Temporal Application. Temporal Blog. Retrieved June 20, 2026 from https://temporal.io/blog/how-to-protect-sensitive-data-in-a-temporal-application.
[39] Sunghwan Yoo, Charles Killian, Terence Kelly, Hyoun Kyu Cho, and Steven Plite. 2012. Composable Reliability for Asynchronous Systems. In *Proceedings of the 2012 USENIX Annual Technical Conference (USENIX ATC ’12)*. USENIX Association.
[40] Roberto Zicari. 2008. Object relational mapping – user case studies. Retrieved November 11, 2025 from https://infoq.com.